

Raritone — Scalability & Cloud Research Notes

Prepared By

BHARATH

VISHNU VARDHAN REDDY

Research Overview

Research was conducted to improve the scalability, cloud readiness, performance, and production stability of the Raritone backend system.

The research focused on:

- **AWS S3 cloud storage**
- **Redis caching**
- **Docker containerization**
- **Load balancing**
- **Logging & monitoring**
- **Image optimization**
- **WebP conversion**
- **Thumbnail generation**
- **Compression planning**

AWS S3 Research

Purpose

AWS S3 is a cloud storage service used for scalable file and image storage.

Benefits for Raritone

- **Scalable image storage**
- **Secure cloud uploads**
- **Backup support**
- **CDN-ready file serving**
- **AI image storage support**

Planned Usage

- **Product images**

- Profile images
- Try-on generated images
- AI output storage

Redis Research

Purpose

Redis is an in-memory caching system used to improve backend performance.

Benefits for Raritone

- Faster API responses
- Reduced MongoDB load
- Improved user experience
- Session management support
- AI request caching

Planned Usage

- Product caching
- Homepage APIs
- JWT blacklist
- User sessions
- Temporary AI data

Docker Research

Purpose

Docker is used to containerize backend services and dependencies.

Benefits for Raritone

- Easy deployment
- Consistent development environment
- Better team collaboration
- Scalable infrastructure
- Cloud deployment support

Planned Containers

- Backend APIs
- MongoDB
- Redis
- AI services
- Image processing services

Load Balancing Research

Purpose

Load balancing distributes traffic across multiple backend servers.

Benefits for Raritone

- Better performance
- Improved server availability
- Reduced server overload
- Scalable traffic handling

Planned Tools

- Nginx
- AWS Load Balancer
- Kubernetes Ingress

Logging & Monitoring Research

Purpose

Logging and monitoring help track backend activity and server health.

Logging Tools

- Morgan
- Winston

Monitoring Tools

- Prometheus
- Grafana
- New Relic

Benefits

- Easier debugging
- API monitoring
- Server health tracking
- Security monitoring
- Production stability

Image Optimization Research

Sharp Library

Sharp is a Node.js image processing library used for image optimization.

Features

- Image resizing
- Compression
- WebP conversion
- Thumbnail generation

Benefits for Raritone

- Faster image loading
- Reduced storage usage
- Better frontend performance
- Optimized try-on image workflows

WebP Conversion Research

Purpose

WebP is an optimized image format that reduces image size while maintaining quality.

Benefits

- Faster frontend loading
- Reduced bandwidth usage
- Improved performance
- Better cloud storage efficiency

Thumbnail Generation Research

Purpose

Generate smaller preview images for product listings and image previews.

Benefits

- **Faster product grid loading**
- **Better user experience**
- **Reduced frontend load time**

Planned Workflow

Original Upload



Generate Thumbnail



Store Both Versions



Use Thumbnail in Product Listings

Compression Planning

Goal

Reduce image size without significantly reducing image quality.

Benefits

- **Faster uploads**
- **Faster frontend loading**
- **Reduced bandwidth usage**
- **Lower cloud storage costs**

Final Summary

Research was successfully completed for:

- **AWS S3**
- **Redis**
- **Docker**
- **Load Balancing**
- **Logging & Monitoring**

- **Sharp Optimization**
- **WebP Conversion**
- **Thumbnail Generation**
- **Compression Planning**

The research prepares the Raritone backend for:

- **Scalable infrastructure**
- **Production-ready deployment**
- **Cloud-based architecture**
- **Optimized image workflows**
- **Future AI integration support**

Additional Section — Cloud Setup Progress

Current Cloud Infrastructure Planning

The Raritone backend is being planned with a scalable cloud-based infrastructure to support:

- **Image-heavy ecommerce workflows**
- **AI try-on processing**
- **Secure uploads**
- **Future backend scaling**

The cloud setup planning focuses on:

- **Cloudinary optimization**
- **Scalable upload handling**
- **CDN-based image delivery**
- **Cloud storage organization**

Cloudinary Workflow Planning

Planned Upload Workflow

Upload



Validate File



Validate File Size



Upload to Cloudinary



Save URL in MongoDB



Return Optimized Response

Planned Cloudinary Folder Structure

Product Images

/products

Profile Images

/profile

Try-On Images

/tryon

Benefits

- **Better scalability**
- **Organized uploads**
- **Easier debugging**
- **Cleaner storage management**

Upload Security Planning

Planned Security Improvements

File Validation

Supported image formats:

- **JPG**
- **PNG**
- **WEBP**

File Size Limits

Maximum planned upload size:

5MB

Benefits

- Prevents backend crashes
- Reduces memory overload
- Optimizes bandwidth usage

CDN Optimization Planning

Why CDN is Important

CDN (Content Delivery Network) improves:

- Faster image loading
- Reduced latency
- Better global performance

Cloudinary CDN support can improve:

- Ecommerce image delivery
- AI try-on image loading
- Frontend performance

Future Cloud Infrastructure Goals

Planned Improvements

- AWS S3 backup storage
- Redis caching
- Docker deployment
- Kubernetes orchestration
- Distributed backend scaling

Production Readiness Goals

The cloud infrastructure planning supports:

- Scalable backend deployment
- Secure image handling
- AI-ready workflows

- **Optimized media delivery**
- **Future enterprise-level scaling**

Final Cloud Setup Summary

Cloud setup research and planning completed for:

- **Cloudinary**
- **Upload workflow optimization**
- **CDN optimization**
- **Scalable cloud storage**
- **Secure upload handling**
- **Production-ready infrastructure planning**

The Raritone backend cloud setup is being planned as a scalable, secure, and production-ready cloud infrastructure system.